Technical Compatibility and Security Report

RoxCrew / MyShift application - deployment assessment for Roxi/Roxer

Assessment date: October 27, 2026
Scope: local repository, runtime configuration, authentication, authorization, persistence, file uploads, real-time behavior and Roxer platform requirements.

EXECUTIVE VERDICT

The application can run as a prototype or staging deployment, but it is not ready for a public production deployment with real user data. Authentication controls, default secrets, session storage, persistence and the serverless/WebSocket model must be addressed first.

Platform assumption

The term 'Roxi' is interpreted as Roxer. Roxer's documentation indicates Node.js 20 and Express support, Lambda-based execution, SQLite persistence at /mnt/data and auto-sleep after inactivity. If a different server is meant, this compatibility matrix must be replaced with that provider's documentation.

Executive summary

Area	Status	Assessment
Node/Express runtime	**OK**	Compatible in principle with Node.js 20 and Express.
Persistence	**CRITICAL**	In-memory Maps plus local JSON; not multi-instance safe.
Uploads	**HIGH**	Local public/uploads storage requires durable storage.
Socket.IO	**HIGH**	WebSocket support in Lambda must be explicitly confirmed.
Security	**CRITICAL**	Authentication bypasses, test account and default secret exist.
Build/runtime	**MEDIUM**	Dependencies are not installed locally; tsx is a dev dependency.

1. Compatibility with Roxi/Roxer

Roxer documents Node.js/Express full-stack hosting through an isolated Lambda function. The current application is not stateless: it keeps state in memory, writes JSON and local files, and creates a persistent HTTP server for Socket.IO.

Component	Current state	Status	Required adaptation
Node/Express	Express server with PORT in server.ts.	**OK**	Pin Node 20 and review build/start commands.
Vite frontend	vite build produces dist.	**OK**	Confirm SPA fallback and static asset serving.
Database	In-memory Maps plus data/database.json.	**CRITICAL**	Migrate to SQLite at /mnt/data or PostgreSQL.
Uploads	multer diskStorage in public/uploads.	**HIGH**	Use S3/R2/object storage or a confirmed durable volume.
Real time	Socket.IO on http.createServer.	**HIGH**	Confirm WebSocket support; otherwise use polling/pub-sub or a persistent server.
Secrets	JWT_SECRET and CLAUDE_API_KEY read from environment.	**HIGH**	Configure Roxer environment variables and remove defaults.
Cold starts	Database and seed state load at startup.	**MEDIUM**	Use idempotent initialization and no production demo seed.

Platform source: Roxer documentation, including runtime, Lambda, SQLite and hosting plan sections.

2. Security findings and required fixes

ID	Severity	Finding and evidence	Required correction
SEC-01	**CRITICAL**	Authentication bypass in **server/middleware.ts:37-51**: x-user-email/x-user-id headers and a default admin user can authenticate requests without a valid JWT.	Remove both fallbacks. requireAuth must accept only a validated HttpOnly cookie or Bearer token.
SEC-02	**CRITICAL**	Known test account and passwords in authRoutes.ts:200-212 and database seeding.	Remove production seed, force credential change and rotate all secrets.
SEC-03	**CRITICAL**	aiRoutes.ts:16 uses roxcrew_claude_secret_2026 as a fallback; the endpoint does not require an authenticated session.	No default value. Require a secret-managed API key, rate limit and board authorization.
SEC-04	**HIGH**	JWT is returned as JSON and stored in localStorage/sessionStorage by src/lib/api.ts:14-21.	Do not expose the token to the frontend; use an HttpOnly cookie.
SEC-05	**HIGH**	Cookie uses SameSite=None in auth.ts:58-65 with no visible CSRF protection.	Use SameSite=Lax/Strict for same-origin use; add CSRF and Origin checks for cross-site use.
SEC-06	**HIGH**	Socket.IO CORS uses origin:true in socket.ts:33-38.	Use an HTTPS allowlist and reject all other origins.
SEC-07	**HIGH**	Invitation routes at invitationRoutes.ts:173-222 check global role but do not consistently enforce tenant/board ownership.	Centralize tenant and board authorization before reading or revoking invitation tokens.
SEC-08	**MEDIUM**	JSON bodies allow up to 50 MB and uploads rely mainly on client-provided MIME types.	Set per-route limits, validate schemas, inspect file signatures and reject unsafe content.

3. Persistence, data and operations

The server loads all records into Maps and serializes them into one JSON file. In a serverless environment, multiple instances may have different copies of the state.

- Migrate users, boards, lists, cards, members, invitations and activity to SQLite at /mnt/data only if Roxer confirms that volume for the app; use managed PostgreSQL for growth.

- Remove data/database.json from Git and create a migration process with encrypted backups.

- Replace saveToDisk with transactions and constraints: unique email, relationships, invitation expiration and concurrency control.

- Move avatars, photos and wallpapers to private object storage with signed, expiring URLs.

- Do not serve uploads from the application filesystem. Preserve board authorization when issuing downloads.

- Add automated backups, retention, tested restoration and audit logs for permission and membership changes.

- Add a health check, error/latency/cold-start/storage metrics and alerts without exposing secrets.

4. Real-time architecture

Socket.IO works well in a persistent Node process, but the reviewed Roxer documentation describes an isolated Lambda function. Persistent WebSocket connections and the in-memory activeBoardUsers map must not be assumed to survive or be shared between invocations.

- Confirm official Roxer support for WebSocket/Socket.IO, timeout, upgrade and sticky sessions.

- If unsupported, replace the design with polling, supported SSE or an external pub/sub provider.

- Do not keep presence in local memory; use Redis, DynamoDB or another shared store if the function scales.

- Test two simultaneous instances: a card change by one user must reach another user.

- Handle reconnects, duplicate events, ordering and idempotency.

5. Detailed remediation plan

Phase	Work	Verifiable result
P0 Security	Remove bypasses, demo account, default secrets, browser token storage and fix cookie/CSRF.	Missing JWT, fake headers and external origin return 401/403.
P0 Data	Migrate JSON and uploads to durable storage; separate demo seed from production.	Restart/redeploy preserve data and two instances see the same records.
P0 Runtime	Pin Node 20, compile backend TypeScript to JavaScript and start with node.	Clean npm install, build and start succeed.
P1 API	Schema validation, per-route limits, rate limits, uniform errors and CORS allowlist.	API suite passes valid, invalid, abuse and tenant permission cases.
P1 Files	Validate file content, names, size, optional malware scan, signed URLs and cleanup.	HTML/SVG executables and disguised payloads are rejected.
P1 Real time	Confirm WebSocket or change architecture; test reconnect and multi-instance behavior.	Events remain consistent after cold starts.
P2 Operations	CI/CD, SAST, dependency audit, backups, alerts, rollback and HTTPS domain.	Release is reproducible, auditable and rollback-tested.

6. Production readiness checklist

Control	Approval criterion	Current state
Secrets	All secrets are Roxer variables; no defaults or Git secrets.	**PENDING**
Authentication	No bypasses; secure cookies; revocable sessions.	**PENDING**
Authorization	Tenant, board, role and file-access tests pass.	**PENDING**
Persistence	DB/files survive restart, redeploy and concurrency.	**PENDING**
Build	Clean Node 20 build/start without dev tools at runtime.	**PENDING**
WebSocket	Support confirmed or alternative implemented.	**PENDING**
Testing	Lint, unit, integration, security and post-deploy smoke tests pass.	**PENDING**
Backups	Encrypted backup and restoration have been validated.	**PENDING**

7. Recommended deployment configuration

The exact configuration depends on the Roxer account, but the minimum expected setup is:

Required environment variables

```
NODE_ENV=production
JWT_SECRET=<high-entropy random secret>
CLAUDE_API_KEY=<only if the endpoint remains>
DATABASE_URL=<if PostgreSQL is used>
STORAGE_BUCKET=<private object-storage bucket>
APP_ORIGIN=https://<official domain>
```

Publication rules

- Do not deploy data/database.json or demo credentials.

- Do not run production with tsx from devDependencies; compile and start with node.

- Use HTTPS and a custom domain only after validating cookies, CORS and WebSocket behavior.

- Use a staging environment separate from production.

- Run smoke tests for login, boards, invitations, uploads, permissions and reconnects.

- Review Roxer logs without recording passwords, JWTs, cookies, API keys or invitation tokens.

8. Local verification result

Static review found the issues listed above. Build verification could not complete because node_modules is not installed: npm run lint and npm run build returned tsc: command not found and vite: command not found. npm audit could not run because package-lock.json is absent; the repository contains bun.lock.

Final conclusion

Do not publish this version with real users. Complete the P0 hardening cycle, migrate persistence, validate WebSocket behavior on Roxer and deploy to isolated staging first. After the checklist passes, the application can be considered for Roxer production if the platform confirms adequate persistence and real-time support.

Sources reviewed

- Roxer documentation: https://roxer.com/docs/ (Node.js 20, Express, Lambda, SQLite at /mnt/data, plans, environment variables and domains).
- Local code reviewed: server.ts, server/auth.ts, server/middleware.ts, server/routes/authRoutes.ts, server/routes/aiRoutes.ts, server/routes/invitationRoutes.ts, server/socket.ts, server/db.ts and src/lib/api.ts.